

(1) Overview

Title

quantum-group: Exact SymPy Verification of Explicit $U_q(\mathfrak{sl}_2)$ and $GL_q(2|1)$ Matrix Identities

Paper Authors

1. Terekli, Taha Berk
ORCID: 0009-0004-8266-1116
Corresponding author: terekli@tahaberk.com

Paper Author Roles and Affiliations

1. Department of Mathematics, Yıldız Technical University, Istanbul, Türkiye **[PENDING: author to confirm current affiliation]**. Sole developer and maintainer of the software; author of this paper.

Abstract

`quantum-group` is an open-source Python package for checking explicit matrix identities that arise in calculations with quantum groups. Identities such as the Yang–Baxter equation, the compatibility of an R-matrix with a coproduct, or the defining relations on a representation are simple to state but error-prone to check by hand, because the answer depends on basis order, the placement of tensor factors, the chosen coproduct and, for superalgebras, sign rules. The package builds the relevant matrices exactly with SymPy, returns the difference between the two sides of an identity as an exact residual matrix, and tests whether every entry simplifies to zero. It covers finite-dimensional $U_q(\mathfrak{sl}_2)$ modules, their tensor products and fundamental R-matrix, and the graded R-matrix of the quantum supergroup $GL_q(2|1)$ of Çelik and Çelik. Researchers and students can use it to check published matrices in their own conventions and to keep those checks as regression tests. It is MIT-licensed, developed on GitHub and archived on Zenodo **[PENDING: archive DOI]**.

Keywords

quantum groups; symbolic computation; SymPy; R-matrix; Yang–Baxter equation; Lie superalgebras; computer algebra; reproducible research

Introduction

Quantum groups are one-parameter deformations of classical symmetry algebras. Their representations yield solutions of the quantum Yang–Baxter equation (QYBE), which is why they appear in exactly solvable models of statistical mechanics, in knot invariants and in representation theory [1, 2, 3, 4]. Much of the working knowledge in this area is recorded as explicit matrices whose entries are rational functions of the deformation parameter q : an R-matrix on $V \otimes V$, the generators of a module, a change of basis to irreducible summands. Before such a matrix can be reused it has to be checked, and checking it means multiplying 9×9 , 27×27 or 81×81 matrices whose meaning depends on conventions that texts state in different ways, or not at all.

The characteristic mistakes are not arithmetic ones. A matrix is transcribed in a different basis order from the one its source used. R_{13} is formed by conjugating with the wrong permutation. An R-matrix is paired with the opposite of the coproduct it actually intertwines; this still satisfies the QYBE, so the most common test does not detect it. In a $\mathbb{Z}_2$ -graded setting an ordinary flip is used where the Koszul sign rule requires a super-permutation. And a check that is run at a floating-point value of q can hide cancellations or produce spurious nonzero entries. Each of these errors produces output that looks plausible.

`quantum-group` addresses this with a deliberately narrow design. Every object is an ordinary exact `sympy.Matrix` [5]; conventions (basis order, coproduct, parities) are fixed in documented functions; and every identity is exposed as a *residual*, the exact matrix LHS – RHS, which is then tested entry by entry. For $U_q(\mathfrak{sl}_2)$ the package constructs the irreducible modules V_n , checks the defining relations and generator-level Hopf identities, builds tensor products through the coproduct, computes Clebsch–Gordan highest-weight vectors, and verifies the fundamental R-matrix: QYBE, braid and Hecke relations and coproduct intertwining. For the quantum supergroup $GL_q(2|1)$ introduced by Çelik and Çelik [6], whose third basis vector is odd, it constructs the published 9×9 R-matrix and verifies the graded Yang–Baxter equation on three factors and on every triple of factors in $V^{\otimes 4}$.

The software was written during the author’s undergraduate thesis [7], where it produced the thesis’s tables, matrix figures and Yang–Baxter verifications. It was later used to audit that thesis: an explicit intertwining residual showed that the thesis’s R-matrix intertwines the opposite of the coproduct used in the text, an error that passing QYBE, braid and Hecke checks had not revealed. The correction is recorded in the repository (`thesis/ERRATA.md`, `AUDIT.md`), and the check is now part of the API and the test suite. No new mathematical results are claimed here; the contribution is the software and its verification method.

The intended users are researchers who need to confirm a matrix identity from the literature in their own conventions, mathematical physicists working with R-matrices, representation theorists and computer-algebra users who want an inspectable reference calculation, and students reproducing standard quantum-group computations.

Related software. GAP’s QuaGroup package [8, 9, 10] and SageMath [11], which provides a `QuantumGroup` interface to it, compute with quantized enveloping algebras $U_q(\mathfrak{g})$ of arbitrary semisimple Lie algebras, including PBW-type bases, highest-weight modules, R-matrices and canonical bases; SageMath also implements crystals for the quantum superalgebras $U_q(\mathfrak{gl}(m|n))$ [12]. These systems are far broader, and for general $U_q(\mathfrak{g})$ work they are the appropriate tools. `quantum-group` does not replace them. It is a small verification layer for concrete matrix calculations: it keeps the basis order and coproduct of a particular text, returns the matrices themselves, runs in an ordinary Python environment, and includes a graded $GL_q(2|1)$ example that lies outside QuaGroup’s documented $U_q(\mathfrak{g})$ scope.

Scope and limitations. A vanishing residual verifies an identity on the given finite-dimensional representation; it is not a proof in the abstract Hopf algebra, and the package is not a theorem prover. It does not implement general quantum groups. For $GL_q(2|1)$ it implements the R-matrix, the graded Yang–Baxter equation and tensor placements, not the RTT relations, the Hopf superalgebra structure or the Gauss decomposition of [6]. It verifies the Hecke relation but does not compute a Markov trace or the Jones polynomial. Its root-of-unity helpers substitute special values into the generic construction; they do not implement the small quantum group. The crystal graph $B(n)$ is a combinatorial model rather than a crystal-basis algorithm.

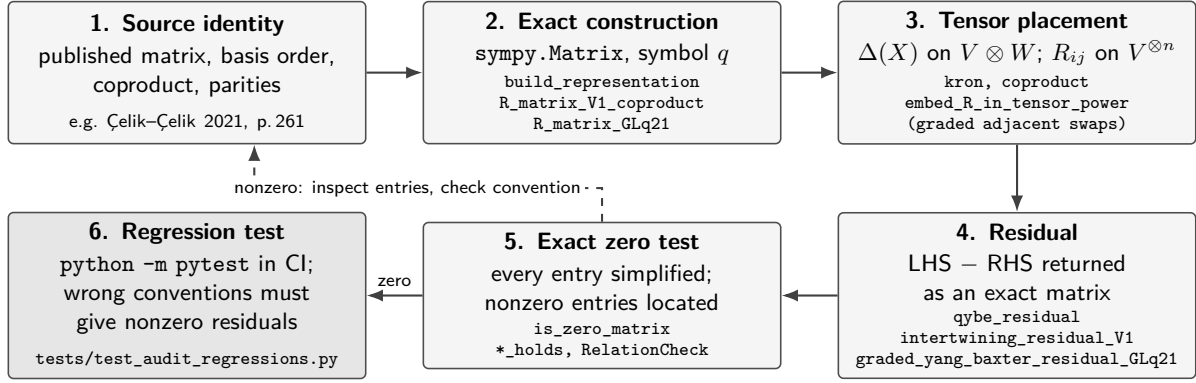


Figure 1: Verification workflow in `quantum-group`. A published identity and its conventions (1) are turned into exact SymPy matrices (2) and placed on tensor products through the coproduct or through graded adjacent swaps (3). The difference of the two sides is returned as an exact residual matrix (4) and tested entry by entry (5); a nonzero residual points to the entries, and hence the convention, that disagree. Zero residuals, and nonzero residuals for deliberately wrong conventions, are kept as regression tests (6). Function names are those of the public API.

Implementation and architecture

`quantum-group` is a pure-Python package of fifteen modules (about 2,000 lines, including docstrings) with no compiled extensions. Figure 1 shows the workflow that the modules implement; the design decisions behind each step are described below.

Explicit matrices rather than an abstract algebra engine. The package contains no noncommutative rewriting system; the symbols in `generators.py` only display the relations. Every check runs on concrete representation matrices. This keeps each object directly comparable, entry by entry, with a printed matrix, and makes every intermediate result an ordinary SymPy object that users can print, substitute into or pass to other SymPy code. The price is that a verification holds for the representations on which it is run, and that the cost grows with the dimension of the tensor product (see *Scaling*). Identities that need a genuinely noncommutative graded algebra, such as the Gauss decomposition of $GL_q(2|1)$, are omitted rather than approximated with commuting symbols.

Layers. The modules form three layers. A small foundation provides the default symbol q (declared nonzero), q -integers, q -factorials and q -binomials (`utils`); the Kronecker product `kron/kron_list` and the zero-residual test `is_zero_matrix` (`linalg`); and internal argument checks (`_validation`). Construction modules build the matrices: $U_q(\mathfrak{sl}_2)$ modules (`representations`), coproduct, counit and antipode (`hopf`), tensor products and Clebsch–Gordan vectors (`tensor`), $U_q(\mathfrak{sl}_2)$ R-matrices (`r_matrix`) and the $GL_q(2|1)$ R-matrix, super-permutation and embeddings (`supergroup_gl21`). Verification functions sit next to the constructions they check and all use the same foundation. Auxiliary modules provide classical and root-of-unity specializations (`limits`), the crystal graph $B(n)$ as a NetworkX [13] graph (`crystal`), Matplotlib [14] weight and crystal plots (`visualization`) and a convenience facade, `QuantumGroupSL2`. Everything public is re-exported from the package namespace and listed in `__all__`.

Residual-first verification. Each check follows one pattern: build the left-hand side, build the right-hand side, return their difference, and decide whether every entry is zero. Functions named `*_residual` (`qybe_residual`, `braid_relation_residual`, `intertwining_residual_V1`, `graded_yang_baxter_residual_GLq21`, and others) return the exact difference matrix; the corresponding `*_holds` functions apply `is_zero_matrix` to it; and `verify_on_representation` returns, for each defining relation, a `RelationCheck` record holding both the Boolean and the residual. Returning the residual rather than only a Boolean is what makes a failed check

informative: the positions and values of the nonzero entries identify which basis vectors and which convention disagree. For example, placing the $GL_q(2|1)$ R-matrix with an ordinary flip instead of the super-permutation leaves exactly two nonzero entries of the 27×27 residual, both equal to $2q^2(q-1)^2(q+1)^2$ (Listing 1). The zero test simplifies each entry with `sympy.simplify`. A `True` result means that every entry was reduced to the exact value 0; a `False` result is not by itself a proof of inequality, because `simplify` is not a decision procedure. For the rational functions of q produced by the package, a nonzero entry can be confirmed with `sympy.cancel`, which the regression tests do.

Exact parameters. The default parameter is a nonzero SymPy symbol, and the identities are rational in q , holding wherever their denominators are defined. Python integers and rationals passed as q are converted to exact SymPy numbers, so `build_representation(2, 2)` yields entries such as $5/2$ rather than 2.5 ; only floating-point values supplied by the user are approximate. q -integers and q -binomials are evaluated as Laurent polynomials, not as quotients, so that specializations with removable singularities, such as $q = -1$ or roots of unity, give the correct values instead of $0/0$.

Representations and tensor products. `build_representation(n)` returns the $(n+1)$ -dimensional module V_n as a `Representation` record with explicit matrices E , F , K and K^{-1} in the basis $v_0, \dots, v_n$, where K is diagonal, F is a lower shift and E carries the products of q -integers. The coproduct is fixed as

$$\Delta(E) = E \otimes 1 + K \otimes E, \quad \Delta(F) = F \otimes K^{-1} + 1 \otimes F, \quad \Delta(K) = K \otimes K,$$

and `coproduct` and `tensor_product` realize it with `kron` in the lexicographic basis order (left factor slow, right factor fast). `find_highest_weight_vectors` solves for the vectors annihilated by E in each weight space of $V_m \otimes V_n$, giving one highest-weight vector per Clebsch–Gordan summand.

Two R-matrix conventions. `R_matrix_V1` returns the historical upper-triangular matrix on $V_1 \otimes V_1$, the image of Drinfeld’s universal R-matrix rescaled by $q^{1/2}$ [1]. It intertwines the opposite of the package coproduct. Rather than change its output, which would silently alter results already computed with it, release 1.1.0 kept it unchanged and added `R_matrix_V1_coproduct` ($R_{21} = PRP$), `R_check_V1_coproduct` and `intertwining_residual_V1`, which do match the package coproduct. Both conventions satisfy the QYBE and the Hecke relation; they differ in which tensor-product submodules the braid eigenspaces are. Keeping both, with the convention stated in each docstring, lets earlier calculations be reproduced exactly while new work uses the compatible matrices. The same policy applies to names: a renamed function keeps its old name as a thin wrapper that emits a `DeprecationWarning`.

Graded tensor embeddings. For $GL_q(2|1)$ the basis parities $(0, 0, 1)$ are an explicit argument, and the graded flip is built once, by `super_permutation_matrix`, as $P_s(e_i \otimes e_j) = (-1)^{p(i)p(j)} e_j \otimes e_i$. `embed_R_in_tensor_power` places a $d^2 \times d^2$ operator on factors (i, j) of $V^{\otimes n}$ by first embedding it on the adjacent pair $(i, i+1)$ with Kronecker products and then conjugating by graded adjacent swaps $S_k = I^{\otimes k} \otimes P_s \otimes I^{\otimes (n-k-2)}$ until the second factor reaches position j . The Koszul sign rule is therefore encoded in a single matrix and reused for every placement, instead of being written out separately for R_{13} , R_{14} , R_{24} and so on. The construction is correct only for even (parity-preserving) operators, and the function checks this before embedding. Because the parity list is an argument, the same function also embeds operators on other super vector spaces, and an all-even parity list reproduces the ungraded placement.

Validation and failure modes. Public functions reject invalid input with specific errors rather than returning a misleading matrix: non-integral or negative highest weights, sizes and indices raise `TypeError` or `ValueError`; `verify_on_representation` rejects non-square or mismatched generator matrices and the singular values $q \in \{0, 1, -1\}$ of the defining commutator quotient; `qybe_residual` rejects matrices that are not $d^2 \times d^2$; and `embed_R_in_tensor_power` rejects

invalid positions, parity lists and odd operators. Symbolic input is not restricted: any SymPy expression can be used as q .

Scaling. Operators on $V^{\otimes n}$ are stored as dense $3^n \times 3^n$ symbolic matrices for $GL_q(2|1)$, which keeps every entry inspectable but grows exponentially. In the recorded benchmark (`benchmarks/results.md`; Python 3.11, SymPy 1.14, 2.1 GHz Xeon, mean of three runs), the graded YBE on $V^{\otimes 3}$ (27×27) takes 0.06 s, the four local YBE checks on $V^{\otimes 4}$ (81×81) 1.6 s, and building all ten placements R_{ij} on $V^{\otimes 5}$ (243×243) 14.6 s. A rerun on 27 September 2026 with Python 3.12.3 on the same processor type gave 1.1 s and 9.7 s for the last two, with a peak resident memory of about 110 MB. $n \geq 6$ has not been benchmarked. Sparse or tensor-network representations would scale further but would hide the matrices that the package exists to show; the target calculations need at most four or five factors.

Quality control

Every change is checked by one command, `python -m pytest`, which runs the test suite with `pytest` [15] together with the package doctests. At the time of writing the suite comprises 198 test cases in ten files and four doctests (202 in total, all passing). GitHub Actions runs it on every push and pull request on Python 3.10, 3.11, 3.12 and 3.13; once with the oldest supported dependency versions (SymPy 1.10.1, NetworkX 2.6.3, Matplotlib 3.5.3 on Python 3.10); and, after building and installing a wheel, runs the README quickstart and the example of Listing 1 from outside the source tree, comparing the latter’s output with a stored copy. A further job repeats the wheel installation, the examples and the full test suite on Linux, macOS and Windows runners **[PENDING: record the result of the first cross-platform CI run]**. Table 1 summarizes what the tests establish; `MANUSCRIPT_CODE_MAPPING.md` in the repository links each computational claim to its implementation and tests.

The negative controls deserve emphasis. A suite that only checks that correct identities hold cannot show that the checks are able to fail. Several tests therefore apply a known wrong convention (the opposite coproduct, the ordinary instead of the graded flip, a mutated counit) and require the residual to be nonzero, in some cases at specified entries with specified values. These tests fix the scientific content of the conventions, not only the behaviour of the code. Two further tests are independent re-derivations: the four-factor placements are compared with a formula that acts on basis vectors directly, and the normalization of the fundamental R-matrix is recomputed from Drinfeld’s universal R-matrix [1].

A user can check an installation in a few seconds with `examples/sample_verification.py`, reproduced as Listing 1 with its output. It runs against the installed package from any directory.

Table 1: Evidence provided by the test suite. Entries in the last column abbreviate `tests/test_name.py`. “Negative control” marks tests that apply a known wrong convention and require a nonzero residual.

Evidence	What is checked	Test file
Exact entries	Every nonzero entry of the 9×9 $GL_q(2 1)$ R-matrix against the display on printed page 261 of [6]; exact entries of the 4×4 $U_q(\mathfrak{sl}_2)$ R-matrix; shift and weight structure of V_n	<code>audit_regressions</code> , <code>r_matrix</code> , <code>representations</code>
Coproduct convention	$R_{21}\Delta(X) = \Delta^{\text{op}}(X)R_{21}$ for $X = E, F, K, K^{-1}$; negative control: the historical matrix fails for E and F	<code>audit_regressions</code>
Clebsch–Gordan bases	Complete change of basis for $V_1 \otimes V_1$, $V_2 \otimes V_2$, $V_3 \otimes V_2$: invertible over $\mathbb{Q}(q)$ and intertwining all four generators; braid eigenvalues q and $-q^{-1}$ on the V_2 and V_0 summands	<code>audit_regressions</code> , <code>tensor</code>
Graded embeddings	All six placements R_{ij} on $V^{\otimes 4}$ equal an independently written signed basis-action formula that builds no swap matrix; disjoint placements commute	<code>audit_regressions</code> , <code>supergroup_gl21</code>
Grading	Graded YBE residual (27×27) is zero; negative control: with the ordinary flip exactly two entries are nonzero, with their values fixed	<code>supergroup_gl21</code> , <code>audit_regressions</code>
Exact parameters	No floating-point atoms for integer, rational, $q = -1$ and $q = i$ inputs; Laurent-polynomial q -integers and q -binomials agree with generic formulas after substitution; root-of-unity orders validated	<code>audit_regressions</code> , <code>limits</code>
Algebraic identities	Defining relations on V_n ; coassociativity, counit and antipode on generators; QYBE, braid and Hecke relations	<code>relations</code> , <code>hopf</code> , <code>r_matrix</code>
Input validation	Every validation path raises the documented error; shared helpers agree with <code>sympy.kronecker_product</code>	<code>validation_and_linalg</code>

```

from quantum_group import (
    build_representation, verify_on_representation, R_matrix_V1_coproduct,
    qybe_holds, intertwining_holds_V1, R_matrix_GLq21,
    embed_R_in_tensor_power, graded_yang_baxter_residual_GLq21,
    is_zero_matrix)

rep = build_representation(2)          # V_2 of U_q(sl_2): 3x3 matrices
print("E on V_2:", rep.E.tolist())
checks = verify_on_representation(rep.E, rep.F, rep.K, rep.K_inv)
print("relations:", {k: c.holds for k, c in checks.items()})

R = R_matrix_V1_coproduct()            # R_21, matches the package coproduct
print("QYBE:", qybe_holds(R))
print("intertwines Delta:",
      [intertwining_holds_V1(X) for X in ("E", "F", "K", "K_inv")])

Y = graded_yang_baxter_residual_GLq21() # GL_q(2|1), parities (0, 0, 1)
print("graded YBE residual:", Y.shape, "zero:", is_zero_matrix(Y))

# Negative control: the same R placed with ordinary (all-even) swaps.
R12, R13, R23 = (embed_R_in_tensor_power(R_matrix_GLq21(), ij, 3, [0, 0, 0])
                  for ij in [(0, 1), (0, 2), (1, 2)])
bad = (R12 * R13 * R23 - R23 * R13 * R12).expand()
print("ungraded residual, nonzero entries:",
      {(i, j): x.factor()
       for (i, j), x in sorted(bad.todok().items()) if x != 0})

```

Listing 1: Sample input: `examples/sample_verification.py`.

Expected output (`examples/sample_verification_expected.txt`; the last line is wrapped here):

```
E on V_2: [[0, q + 1/q, 0], [0, 0, q + 1/q], [0, 0, 0]]
relations: {'R1': True, 'R2': True, 'R3': True, 'R4': True}
QYBE: True
intertwines Delta: [True, True, True, True]
graded YBE residual: (27, 27) zero: True
ungraded residual, nonzero entries: {(8, 24): 2*q**2*(q - 1)**2*(q + 1)**2,
  (17, 25): 2*q**2*(q - 1)**2*(q + 1)**2}
```

The same output was obtained with SymPy 1.14 on Python 3.12 and with SymPy 1.10.1 on Python 3.10. Documentation consists of the README (installation, quickstart, conventions, limitations), docstrings stating the convention and relation of every public function, CONTRIBUTING.md, CHANGELOG.md and an exploratory Jupyter notebook.

(2) Availability

Operating system

Any operating system with CPython 3.10 or later; the package is pure Python. Tested in continuous integration on Linux (GitHub Actions `ubuntu-latest`) and locally on Ubuntu 24.04 (x86-64). **[PENDING: macOS and Windows: state as tested only after the cross-platform CI job has passed]**

Programming language

Python 3.10 or later (`requires-python = ">=3.10"`); tested on Python 3.10, 3.11, 3.12 and 3.13.

Additional system requirements

None beyond a standard Python installation; no GPU, network access or special hardware is needed. Run time and memory grow exponentially with the number of tensor factors: the largest benchmarked case (243×243 symbolic matrices) takes about 10–15 s on one core of a 2.1 GHz server processor with a peak resident memory of about 110 MB. Larger tensor powers have not been benchmarked.

Dependencies

Runtime (installed automatically): SymPy ≥ 1.10 [5], NetworkX ≥ 2.6 [13] and Matplotlib ≥ 3.5 [14]; these lower bounds are exercised in continuous integration. Optional: pytest for the tests (extra `[test]`) and Jupyter for the notebook (extra `[notebooks]`). Building from source uses `setuptools` ≥ 77 and `wheel`.

List of contributors

Taha Berk Terekli, Department of Mathematics, Yıldız Technical University: design, implementation, testing, documentation and maintenance. No other person has contributed code. Generative AI tools were used during publication preparation; they are not contributors and their use is disclosed below.

Software location

Archive

Name:

Zenodo

Persistent identifier:

[PENDING: version DOI of the v1.1.0 archive; the existing DOI 10.5281/zenodo.22997681 archives v1.0.0 only and must not be used]

Licence:

MIT

Publisher:

Taha Berk Terekli

Version published:

1.1.0

Date published:

[PENDING: release date of v1.1.0]

Code repository

Name:

GitHub

Identifier:

<https://github.com/TerekliTahaBerk/quantum-groups>

Licence:

MIT

Date published:

[PENDING: release date of v1.1.0] (repository created 24 March 2026)

Package registry

Name:

PyPI

Identifier:

[PENDING: <https://pypi.org/project/quantum-group/> once published; install with `python -m pip install quantum-group==1.1.0`]

Language

English (source code, documentation and repository). Historical thesis material in the `thesis/` directory is in Turkish.

(3) Reuse potential

`quantum-group` is small enough to read in full, and its extension points are ordinary Python functions that accept and return SymPy matrices. The following uses are supported by the current API; for each, the route to reuse is stated.

Checking a published R-matrix transcription. A researcher who types an R-matrix from a paper into a `sympy.Matrix` can call `qybe_residual` (or `braid_relation_residual` for a braided form)

on any $d^2 \times d^2$ matrix. A nonzero residual locates the entries where the transcription or the source is inconsistent. For graded R-matrices, the parities are passed to `embed_R_in_tensor_power`, and the three placements R_{12} , R_{13} , R_{23} are multiplied exactly as in Listing 1.

Deciding which coproduct an R-matrix belongs to. `intertwining_residual_V1` shows the pattern for $V_1 \otimes V_1$: build $\Delta(X)$ with `coproduct`, conjugate by `swap_matrix` to obtain $\Delta^{\text{op}}(X)$, and form $R\Delta(X) - \Delta^{\text{op}}(X)R$. Replacing the matrix, the module or the coproduct formulas in `hopf.coproduct` gives the same test for another convention; this is the check that exposed the convention error in the author’s thesis.

Turning literature identities into regression tests. Because each check returns an exact matrix, an identity from a paper or thesis becomes a pytest test in a few lines, and a known wrong convention can be kept alongside it as a negative control. `tests/test_audit_regressions.py` contains worked examples, including source-entry tests and independently derived comparison formulas. Projects that develop their own mathematical software can adopt the same pattern as a guard against convention drift.

Other finite-dimensional representations. The residual pattern does not depend on $U_q(\mathfrak{sl}_2)$. `verify_on_representation` accepts any four square matrices, so alternative bases or normalizations of V_n can be checked directly. Adding a further explicit R-matrix, for example for a higher-dimensional module or another rank-one example, needs only a function returning its matrix: `kron`, `qybe_residual`, `is_zero_matrix` and `embed_R_in_tensor_power` are reused unchanged.

Other super vector spaces. `super_permutation_matrix` and `embed_R_in_tensor_power` take an arbitrary parity list, not only $(0, 0, 1)$. They can place any even operator on factors (i, j) of $V^{\otimes n}$ for other $\mathbb{Z}_2$ -graded spaces, such as $\mathbb{C}^{m|n}$ with a different ordering of even and odd basis vectors, with the sign rule applied consistently.

Teaching and reproducing calculations. Tensor products, coproduct actions and Yang–Baxter products are ordinary matrices that can be printed at every step, which makes the package usable for exercises on how basis order and coproduct choice affect a calculation. For theses and papers, a short script that calls the package can accompany a convention-sensitive calculation so that readers can rerun it.

Where reuse requires new work. Some extensions are not incremental. Moving from $\mathfrak{sl}_2$ to general semisimple Lie algebras would need root data, PBW bases and higher-rank modules, which QuaGroup and SageMath already provide. Implementing the RTT relations or the Hopf superalgebra of $GL_q(2|1)$ would need a noncommutative graded algebra rather than matrices. The dense matrix representation limits practical calculations to about five tensor factors of a three-dimensional space. These directions are possible but would be substantial new software, not reuse of the existing layer.

Integration. Outputs are standard `sympy.Matrix` objects, so results can be passed to other SymPy code, exported as LaTeX with `sympy.latex`, or compared against matrices computed elsewhere, for example exported from SageMath, by forming a residual.

Support and contributions. Questions, bug reports and feature requests are handled through the GitHub issue tracker (<https://github.com/TerekliTahaBerk/quantum-groups/issues>). `CONTRIBUTING.md` describes the development set-up, the validation command and the conventions for new checks (a `*_residual` function with a `*_holds` companion, input validation with a test for each error path, and a stated reference for any convention). The package is maintained by its author on a best-effort basis; there is no institutional support or guaranteed response time. Changes are recorded in `CHANGELOG.md` and versions follow semantic versioning.

Use of generative AI

Generative AI tools were used in preparing the software release and this paper; they are not authors or contributors. The core implementation (April–June 2026) was written by the author during the thesis work **[PENDING: author to confirm whether any AI tool was used in that period]**. From September 2026, Claude Code (Anthropic) **[PENDING: insert the exact model name(s) and version(s), as recorded in the Co-Authored-By trailers of the corresponding commits]** assisted with packaging and continuous integration, the benchmark script, the shared matrix helpers and input validation with their tests, documentation, and drafting of this manuscript and its submission materials; commits produced with it are authored or co-authored by “Claude” in the Git history. Codex (OpenAI) **[PENDING: insert model name and version; not verifiable from the repository]** assisted with translating Turkish docstrings and comments into English and with a mathematical and technical audit that led to the parameter-handling corrections, the coproduct-compatible R-matrix functions and the regression tests described above. AI-assisted code is covered by the automated tests, and AI-assisted mathematical statements were checked with executable symbolic tests and against the primary sources, including the original Çelik–Çelik article. The author reviewed and validated all AI-assisted output, made the scientific and design decisions, and takes responsibility for the software and this paper.

Acknowledgements

The author thanks Prof. Dr. Salih Çelik for supervising the undergraduate thesis from which this software originated.

Funding statement

[PENDING: author to state whether the work received funding; if not, use the journal’s wording for no funding]

Competing interests

[PENDING: author to confirm; if none: “The author has no competing interests to declare.”]

References

1. Drinfeld VG. Quantum groups. *Proceedings of the International Congress of Mathematicians, Berkeley, California, 1986*. Vol. 1. Providence, RI: American Mathematical Society, 1987. p. 798–820
2. Jimbo M. A q -difference analogue of $U(\mathfrak{g})$ and the Yang–Baxter equation. *Letters in Mathematical Physics*. 1985; 10:63–9. DOI: 10.1007/BF00704588
3. Jimbo M. A q -analogue of $U(\mathfrak{gl}(N + 1))$, Hecke algebra, and the Yang–Baxter equation. *Letters in Mathematical Physics*. 1986; 11:247–52. DOI: 10.1007/BF00400222
4. Kassel C. Quantum Groups. Graduate Texts in Mathematics 155. New York: Springer, 1995. DOI: 10.1007/978-1-4612-0783-2

5. Meurer A, Smith CP, Paprocki M, Čertík O, Kirpichev SB, Rocklin M et al. SymPy: symbolic computing in Python. *PeerJ Computer Science*. 2017; 3:e103. DOI: 10.7717/peerj-cs.103
6. Çelik S and Çelik SA. A new quantum supergroup and its Gauss decomposition. *Reports on Mathematical Physics*. 2021; 88:259–69. DOI: 10.1016/S0034-4877(21)00073-2
7. Terekli TB. Quantum grup yapılarının Python ortamında modellenmesi [Modelling quantum group structures in Python]. Undergraduate thesis. Istanbul: Department of Mathematics, Yıldız Technical University, 2026 Jun. Available from: <https://github.com/TerekliTahaBerk/quantum-groups/tree/main/thesis>
8. The GAP Group. GAP – Groups, Algorithms, and Programming. Version 4.16.1. 2026. Available from: <https://www.gap-system.org>
9. de Graaf WA. QuaGroup: computations with quantum groups (GAP package). Version 1.8.4. 2024. Available from: <https://github.com/gap-packages/quagroup>
10. de Graaf WA. Computing with quantized enveloping algebras: PBW-type bases, highest-weight modules and R -matrices. *Journal of Symbolic Computation*. 2001; 32:475–90. DOI: 10.1006/jscs.2001.0479
11. The Sage Developers. SageMath, the Sage Mathematics Software System. Version 10.9. 2026. DOI: 10.5281/zenodo.8042260. Available from: <https://www.sagemath.org>
12. Benkart G, Kang SJ and Kashiwara M. Crystal bases for the quantum superalgebra $U_q(\mathfrak{gl}(m, n))$. *Journal of the American Mathematical Society*. 2000; 13:295–331. arXiv: [math/9810092](https://arxiv.org/abs/math/9810092)
13. Hagberg AA, Schult DA and Swart PJ. Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy 2008)*. Ed. by Varoquaux G, Vaught T and Millman J. Pasadena, CA, 2008. p. 11–5. Available from: https://conference.scipy.org/proceedings/SciPy2008/paper_2/
14. Hunter JD. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*. 2007; 9:90–5. DOI: 10.1109/MCSE.2007.55
15. Krekel H, Oliveira B, Pfannschmidt R, Bruynooghe F, Laughner B, Bruhin F et al. pytest: simple powerful testing with Python. 2004. Available from: <https://github.com/pytest-dev/pytest>